

FI360 CORE ARCHITECTURE & COMMON SERVICES SPECIFICATION

Fleet Intelligence 360 (FI360) — Version 1.0
Fleet Associates Africa Ltd (FAA)
9 August 2026

Document Control

Field	Value
Document ID	FI360-CACS-002
Version	1.0
Status	Baseline / Technical Foundation
Parent	FI360 Master Architecture & Development Framework v1.1
Owner	Fleet Associates Africa Ltd
Date	9 August 2026

1. Purpose

This document defines the FI360 Core: the shared technical and platform foundation that allows independently deployable FI360 modules to operate consistently and connect securely. It is the controlling specification for identity, tenancy, access control, module registration, entitlements, common references, workflow, notifications, audit, APIs, events and platform observability.

2. Architectural Position

The FI360 Core is not a replacement for the business modules. It supplies reusable platform capabilities so that specialist modules do not duplicate identity, security, licensing and integration functions.

FI360 Core Common Services

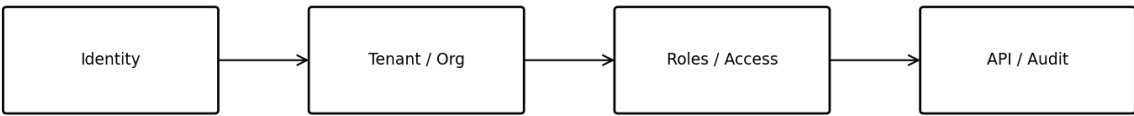


Figure 1. FI360 Core Common Services

3. Core Principles

- Centralise common platform services; decentralise specialist business logic.
- No direct production database coupling between modules.
- Stable identifiers across the platform.
- Security and tenant isolation at every service boundary.
- Independent module activation and later integration.
- APIs/events for controlled communication.
- Auditability, observability and recoverability by design.
- Configuration over customer-specific code where practical.

4. Core Services

Service	Responsibility	Consumers
Tenant Management	Customer/tenant creation, status and isolation	All modules
Identity	Authentication and user identity	All modules
Authorization	Roles, permissions and access scope	All modules
Organisation/Site	Entities, branches, depots and departments	All modules
Module Registry	Available, installed and versioned modules	Platform
Entitlement	Subscription and module activation	Platform/modules
Reference Data	Controlled shared codes and lists	All modules
Workflow	Reusable states, approvals and assignments	Modules
Notifications	In-app/email/SMS/push abstraction	Platform/modules
Audit	Traceable platform and business actions	All modules
API Gateway	API security, routing and controls	Integrations
Event Service	Asynchronous event publication/delivery	Modules/analytics
Configuration	Tenant/module settings and feature flags	Platform/modules
Observability	Logs, metrics, traces, health and alerts	Operations

5. Tenant Architecture

A tenant represents a customer organisation or controlled customer environment. Every tenant-scoped record must carry or inherit tenant context. Requests must never cross tenant boundaries without explicit authorization.

Level	Example	Purpose
Tenant	FI360-TEN-000001	Customer boundary
Organisation	FI360-ORG-000010	Operating/legal entity
Site	FI360-SIT-000021	Depot/branch/workshop
Department	FI360-DEP-000031	Operational unit
User	FI360-USR-000045	Person/system identity
Role	FI360-ROL-000012	Permission bundle

6. Identity and Access Management

FI360 shall provide a common authentication foundation so users do not need separate credentials for each FI360 module.

Identity and Access Flow

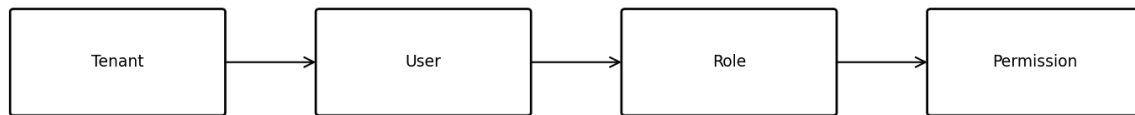


Figure 2. Tenant → User → Role → Permission Flow

6.1 Authentication

- Secure session/token management.
- MFA capability for appropriate deployments.
- Session expiry/revocation.
- Service-to-service authentication.
- No credentials embedded in source code.

6.2 Authorization

- RBAC.
- Module-level permissions.
- Site/branch/department scope.
- Action-level permissions for sensitive functions.
- Deny-by-default for protected resources.

7. Module Registry and Entitlements

The Module Registry is authoritative for knowing which FI360 modules are available, installed, enabled, versioned and supported for each tenant.

Module Registration and Activation

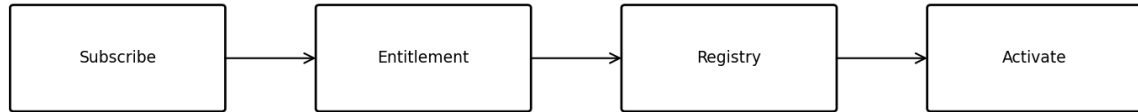


Figure 3. Module Registration and Activation

Object	Key fields
Module	ID, name, version, status, API version
Entitlement	Tenant, module, plan, dates, status, limits
Feature Flag	Tenant/module, feature, enabled state
Installation	Environment, version, deployment status
Dependency	Module, required service/version, compatibility

8. Common Reference Data

Common reference data should be governed centrally only where it is genuinely shared. Specialist codes remain owned by their business module.

- Stable identifiers for reference codes.
- Version changes that affect historical meaning.
- Tenant-specific configuration where justified.
- Historical transactions retain their original code meaning.

9. Canonical Asset Reference

The FI360 Core provides the common reference mechanism for assets. The Fleet Register remains authoritative for the asset master; other modules reference the canonical FI360 Asset ID.

Canonical Asset Reference

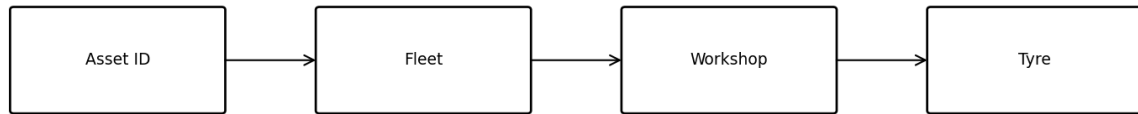


Figure 4. Canonical FI360 Asset Reference

Entity	Owner	Example
Asset master	Fleet Register	FI360-AST-000123
Work order	Workshop	FI360-WO-000458 → AST-000123
Tyre	Tyre	FI360-TYR-000321 → AST-000123
Fuel	Fuel	FI360-FUEL-000789 → AST-000123
Telematics	Telematics	Device event → AST-000123

10. Workflow Service

The Core workflow service provides reusable workflow capabilities without owning specialist business decisions.

Capability	Example
State model	Draft → Submitted → Approved → In Progress → Closed
Assignment	User/team/site/role
Approval	Single or multi-level
SLA	Due date, escalation, breach
Transition	Who may change state
Evidence	Documents/comments

11. Notification Service

- In-app notifications.
- Email/SMS/push through approved providers.
- Templates.
- Event-triggered messages.
- User preferences.
- Delivery status and retry handling.

12. API Gateway and Integration

The API Gateway is the controlled boundary between FI360 modules/services and external systems. It applies authentication, authorization, validation, routing, rate controls and traceability.

API Gateway Flow

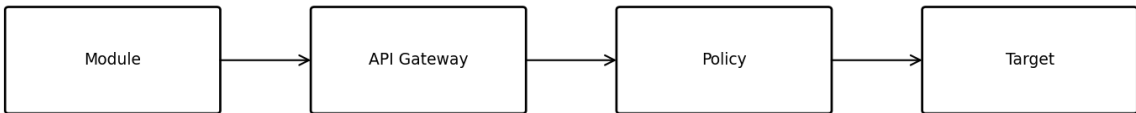


Figure 5. FI360 API Gateway Flow

Control	Requirement
Authentication	Every protected request authenticated
Authorization	Tenant/module/action access checked
Validation	Schema and preconditions validated
Versioning	Explicit supported API versions
Rate control	Controlled traffic
Traceability	Correlation/request ID
Audit	Material actions recorded
Errors	Standard error structure/codes

13. Event Service

Events enable loosely coupled integration. An event represents a business fact that has occurred, such as TYRE_FITTED or WORK_ORDER_COMPLETED.

Event Flow

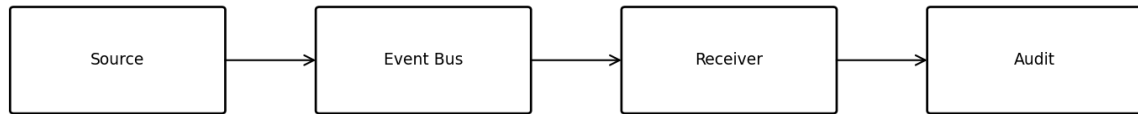


Figure 6. FI360 Event Flow

Attribute	Purpose
Event ID	Unique event
Event Type	Business event name
Version	Schema version
Tenant ID	Tenant context
Entity ID	Affected entity
Timestamp	Occurrence time
Source	Publishing module
Correlation ID	Related transaction
Payload	Approved data

14. Audit and Observability

Material platform and business actions shall be traceable and protected from ordinary user alteration.

- Actor identity.
- Affected entity.
- Action.
- Timestamp.
- Source module/service.
- Correlation ID.
- Relevant before/after values.
- Outcome.

Observability	Examples
Logs	Application, security and integration errors
Metrics	Latency, throughput, queue depth, uptime
Traces	Cross-service request tracing
Health	Service/dependency health
Alerts	Repeated failures, downtime, abnormal processing

15. Configuration and Feature Management

- Tenant settings.

- Module settings.
- Site settings where required.
- Feature flags.
- Environment configuration.
- Secure secrets handling.
- Configuration audit.

Feature flags must not bypass security or licensing controls.

16. Document Service

The Core may provide document metadata and secure storage integration while the owning module controls the business relationship to each document.

- Document ID/metadata.
- Entity association.
- Versioning.
- Access control.
- Retention/deletion policy.
- Security scanning where supported.

17. Data Architecture

The preferred architecture is module-owned transactional data, controlled shared references and a separate analytics/data integration layer. The Core should not become one database containing every module's business tables.

Data	Authoritative owner
Tenant/User/Role	FI360 Core
Asset master	Fleet Register
Workshop transactions	Workshop
Tyre lifecycle	Tyre
Fuel transactions	Fuel
Parts inventory	Parts
Cross-module analytics	Analytics platform
Audit	FI360 Core/audit platform

18. Service-to-Service Communication

- Synchronous APIs where immediate response is required.
- Asynchronous events for loosely coupled processes.
- Controlled retries/backoff.
- Idempotent consumers.
- Dead-letter/error handling.
- Correlation IDs across transactions.

19. Security Architecture

Control	Minimum requirement
Tenant isolation	Mandatory for tenant-scoped resources
Access control	RBAC + scope
Secrets	External secret management

Transport	Encrypted protected traffic
Audit	Security-sensitive actions logged
API security	Authenticated and authorized
Dependencies	Controlled versions/vulnerability management
Recovery	Defined backup/restore and recovery objectives

20. Backup, Recovery and Availability

- Automated backups where applicable.
- Restore testing.
- Defined RPO/RTO per service tier.
- Health checks.
- Graceful degradation for non-critical services.
- Documented disaster recovery.

21. External Integrations

Category	Examples
Telematics	GPS, odometer, engine data
ERP/Finance	Accounting, procurement, asset finance
Payments	Approved payment/mobile money
Fuel	Fuel cards/dispensing
Identity	Enterprise identity providers
Mapping	Maps/geocoding/routing
Regulatory	Approved government interfaces

Each external integration requires its own connector/API contract, credentials, ownership, monitoring and failure-handling design.

22. Multi-Module Example: Tyre + Workshop

A customer may start with FI360 Tyre and later add FI360 Workshop. The Core preserves the same tenant, users and Asset IDs. Tyre remains authoritative for tyre lifecycle; Workshop remains authoritative for work orders.

1. Tenant is identified by FI360-TEN ID.
2. Vehicle is referenced by FI360-AST ID.
3. Workshop creates a work order.
4. Workshop uses the approved Tyre API/event interface.
5. Tyre processes the tyre transaction.
6. Tyre publishes the relevant event.
7. Workshop records the event against its work order.
8. Audit and analytics receive traceable records.

23. Non-Functional Requirements

Area	Principle
Security	Secure by design; tenant isolation
Performance	Predictable response under normal load
Scalability	Scale stateless services where practical
Maintainability	Documented/versioned services
Observability	Logs, metrics, traces, health
Compatibility	Backward-compatible evolution where supported
Recoverability	Backup/restore and DR

Auditability	Material actions traceable
--------------	----------------------------

24. Initial Core Components

Component	Purpose
Identity Service	Authentication/identity
Authorization Service	Roles/permissions/policy
Tenant Service	Tenant/org/site structure
Module Registry	Module catalogue/status
Entitlement Service	Subscription/activation
Configuration Service	Settings/feature controls
Workflow Service	Workflow/approvals
Notification Service	Communication abstraction
Audit Service	Audit records
API Gateway	API boundary/policies
Event Bus/Broker	Asynchronous events
Observability Stack	Logs/metrics/traces/alerts

25. Environments

Environment	Purpose
Development	Engineering/component testing
QA/Test	Integration/regression
UAT	Business acceptance
Staging	Production-like validation
Production	Live service

Environment credentials and configuration must be separated. Production data should not be copied into lower environments without approved protection and masking.

26. Governance Rules

- Core APIs are controlled interfaces.
- Breaking changes require versioning and migration planning.
- Security-impacting changes require review.
- Changes affecting tenant isolation require architecture review.
- New shared services must demonstrate reuse value.
- Specialist business logic remains in modules.

27. Implementation Sequence

9. Define tenant, organisation, site and user schemas.
10. Define authentication and authorization.
11. Implement Module Registry and entitlements.
12. Define canonical Asset ID contract.
13. Implement API Gateway and service authentication.
14. Define audit/correlation standards.
15. Implement event contract/broker pattern.
16. Implement configuration, notification and workflow foundations.
17. Implement observability.
18. Integrate FI360 Tyre.
19. Validate Tyre → Workshop integration.

28. Acceptance Criteria

- Tenant can be created and isolated.
- User can authenticate and access authorised modules.
- Permissions restrict module and scope access.
- Module can be registered, entitled and activated.
- Asset ID can be referenced across modules without duplicate ownership.
- Protected API call is authenticated, authorized and traceable.
- Event can be published, delivered, retried and traced.
- Material actions generate audit records.
- Core health/observability is available.
- Tyre can operate independently and later connect to Workshop.

29. Next Document

DOCUMENT 03 — FI360 MODULE DEVELOPMENT STANDARD will convert the Master Architecture and Core Architecture into a repeatable blueprint for every module. It will define the standard module SRS, boundaries, UI/UX, database conventions, API contract, event catalogue, testing, security, deployment and developer backlog.

Appendix A — Core-to-Module Boundary

FI360 Core owns	Modules own
Tenant identity	Specialist business transactions
Users/authentication	Module-specific business rules
Roles/permissions	Domain workflows
Module registration	Domain data
Entitlements	Domain reports
Common references	Domain calculations
Audit infrastructure	Domain event semantics
API/event infrastructure	Specialist integrations

“FI360 Core provides the common foundation; FI360 modules provide the business capability.”